

微處理器系統實驗報告 - Lab 07

資工二

113590051 楊承諭

113590017 黃楷程

2026 年 5 月 29 日

實驗基本資訊

- 系級：資工二
 - 學號/姓名：113590051 / 楊承諭、113590017 / 黃楷程
 - 日期：2026/05/29
 - 實驗名稱：八位元除法器 (8-Bit Restoring Divider) 硬體設計
-

組員貢獻比例

姓名	學號	貢獻比例	負責內容
楊承諭	113590051	100%	小組專案製作與小組報告製作
黃楷程	113590017	0%	無貢獻內容

1 實驗內容

本次實驗的主題為「八位元恢復除法器 (8-Bit Restoring Divider) 之硬體設計」。實驗目標是使用 VHDL 語言在 FPGA 開發板 (Altera DE2-115) 上，以「結構化 (Structural)」架構實現一個完整的 8 位元二進位除法器，包含資料路徑 (Datapath) 與控制器 (Controller) 兩大部分。

1.1 實驗目標

1. 理解恢復除法演算法 (Restoring Division Algorithm) 的數學原理與流程。
2. 設計並整合一個六狀態的有限狀態機 (FSM) 作為控制器，依序驅動各暫存器的運算。
3. 重複使用 Lab 05 中設計的泛用移位暫存器 (N_bit_DFF) 作為商數 (Q)、除數 (D) 及餘數 (R) 三個暫存器的元件，充分體現硬體的模組化設計精神。

4. 在運算完成後 (S4 狀態)，透過四個七段顯示器 (HEX3 ~ HEX0) 以十六進位顯示最終的商與餘數。

1.2 硬體架構概述

整體設計分為以下兩層架構：

- 資料路徑 (Datapath)：由三個以 `n_bit_DFF` 泛型實例化的暫存器構成。
 - **Q** 暫存器 (商數, **8-bit**)：用於累積商數位元，每次迭代向左移位並串入 `sdi_Q` (S2a 串入 '1', S2b 串入 '0')。
 - **D** 暫存器 (除數, **17-bit**)：初始載入除數於高位元組 (位元 15:8)，每次 S3 狀態向右移一位，模擬二進位對齊。
 - **R** 暫存器 (餘數, **17-bit**)：初始載入被除數於低位元組 (位元 7:0)，每次 S1 載入減法結果，S2b 時恢復 ($R + D$)。
- 組合邏輯 **ALU**：一個 17-bit 減法器 (`sub_res <= qo_R - qo_D`)，其最高符號位元 (`sub_res(16)`) 決定 FSM 由 S1 跳轉至 S2a 或 S2b。

1.3 腳位分配 (Pin Assignment)

Table 1: FPGA 腳位分配表

訊號名稱	方向	FPGA 腳位	功能描述
Clock	in	PIN_M23 (KEY[0])	系統時脈 (手動步進按鍵)
Reset	in	PIN_Y24 (SW[16])	非同步重置，重回 Start 狀態
Divisor	in	SW[7..0]	8-bit 除數輸入
Dividend	in	SW[15..8]	8-bit 被除數輸入
Quotient	out	LEDR[7..0]	即時商數 LED 顯示
Remainder	out	LEDR[15..8]	即時餘數 LED 顯示
HEX0	out	HEX0[6..0]	最終商數低位 nibble (七段顯示)
HEX1	out	HEX1[6..0]	最終商數高位 nibble (七段顯示)
HEX2	out	HEX2[6..0]	最終餘數低位 nibble (七段顯示)
HEX3	out	HEX3[6..0]	最終餘數高位 nibble (七段顯示)

2 邏輯設計與實驗過程

2.1 恢復除法演算法說明

恢復除法演算法是一種基於逐位元 (bit-by-bit) 試減的二進位除法方法，其核心思想為：

1. 初始化 (**Start**)：將除數 B 載入 D 暫存器高位元組，被除數 A 載入 R 暫存器低位元組，Q 暫存器清零。
2. 試減 (**S1**)：計算 $R - D$ ，若結果 ≥ 0 (符號位元為 0)，進入 S2a；否則進入 S2b。
3. **S2a** (不恢復)：餘數更新為減法結果，Q 左移並串入 '1'。

4. **S2b** (恢復)：餘數恢復為 $R + D$ ， Q 左移並串入 '0'。
5. 移位計數 (**S3**)： D 暫存器右移一位 (相當於除數位元對齊下移)。若已完成 9 次迭代，進入 **S4**；否則回到 **S1**。
6. 完成 (**S4**)：凍結所有暫存器，於七段顯示器顯示最終商與餘數。

2.2 有限狀態機 (FSM) 狀態轉移

Table 2: FSM 狀態轉移表

現在狀態	條件	下一狀態
Start	Reset 解除後第一個時脈觸發	S1
S1	<code>sub_res(16) = '0'</code> (結果 ≥ 0)	S2a
S1	<code>sub_res(16) = '1'</code> (結果 < 0)	S2b
S2a	無條件	S3
S2b	無條件	S3
S3	<code>count < 8</code>	S1 (繼續迭代)
S3	<code>count = 8</code>	S4 (完成)
S4	無條件 (直到 Reset)	S4

2.3 實驗步驟

- 在 Quartus 中建立專案，加入 `Division.vhd` 與 `N_bit_DFF.vhd` 兩個設計檔案，並設定 `Division` 為頂層模組。
- 進行編譯與合成，確認無語法及邏輯錯誤。
- 依照腳位分配表完成 Pin Assignment，設定 `SW[15..0]`、`KEY[0]`、`SW[16]`、`LEDR[15..0]` 及 `HEX[3..0]` 腳位。
- 將設計燒錄至 DE2-115 開發板，設定 `SW[15..8]` (被除數) 與 `SW[7..0]` (除數)，然後透過 `SW[16]` 進行 Reset，再以手動按壓 `KEY[0]` 逐步驅動 FSM 進行除法運算。
- 驗證 LED 與七段顯示器的輸出是否正確。

3 實驗結果 (照片紀錄)

以下照片記錄了在 DE2-115 開發板上驗證 8 位元除法器的完整過程，測試案例為 $A \div B = 65 \div 13 = 5 \dots 0$ (即 $0x41 \div 0x0D = 0x05$ 餘 $0x00$)。

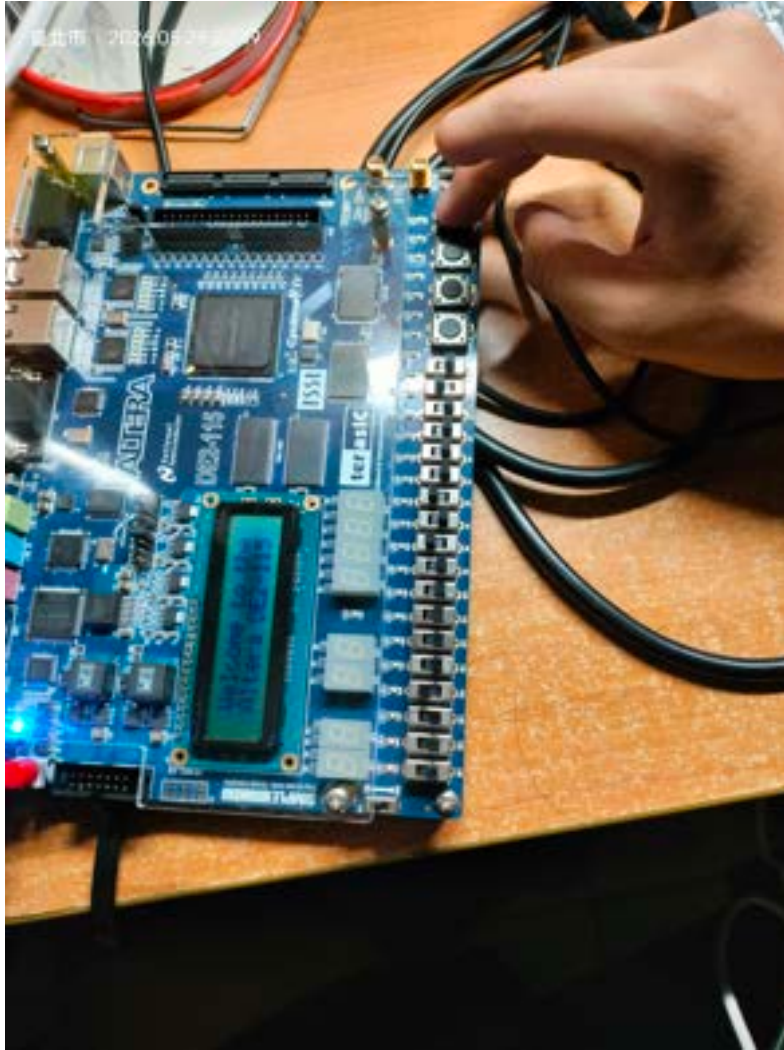


Figure 1: 初始狀態 (Start)：SW[16] 撥高完成 Reset，所有 LED 熄滅，七段顯示空白，準備開始除法運算。

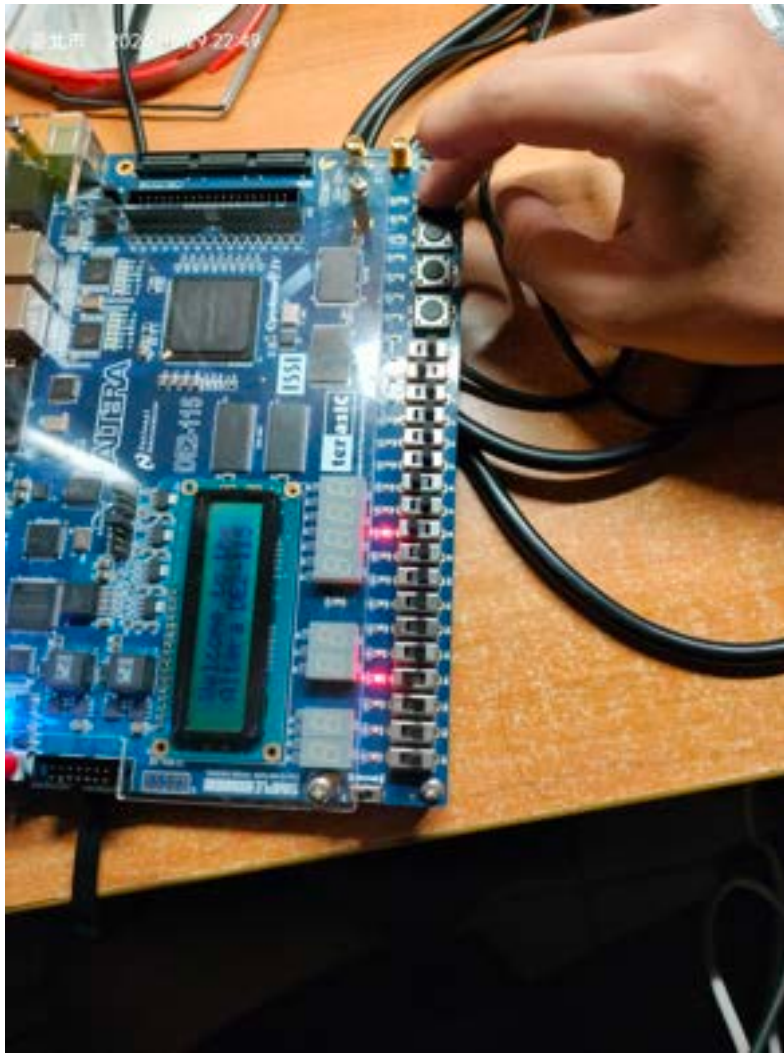


Figure 2: 輸入被除數與除數：SW[15..8] 設定被除數，SW[7..0] 設定除數，確認撥碼開關設定正確。

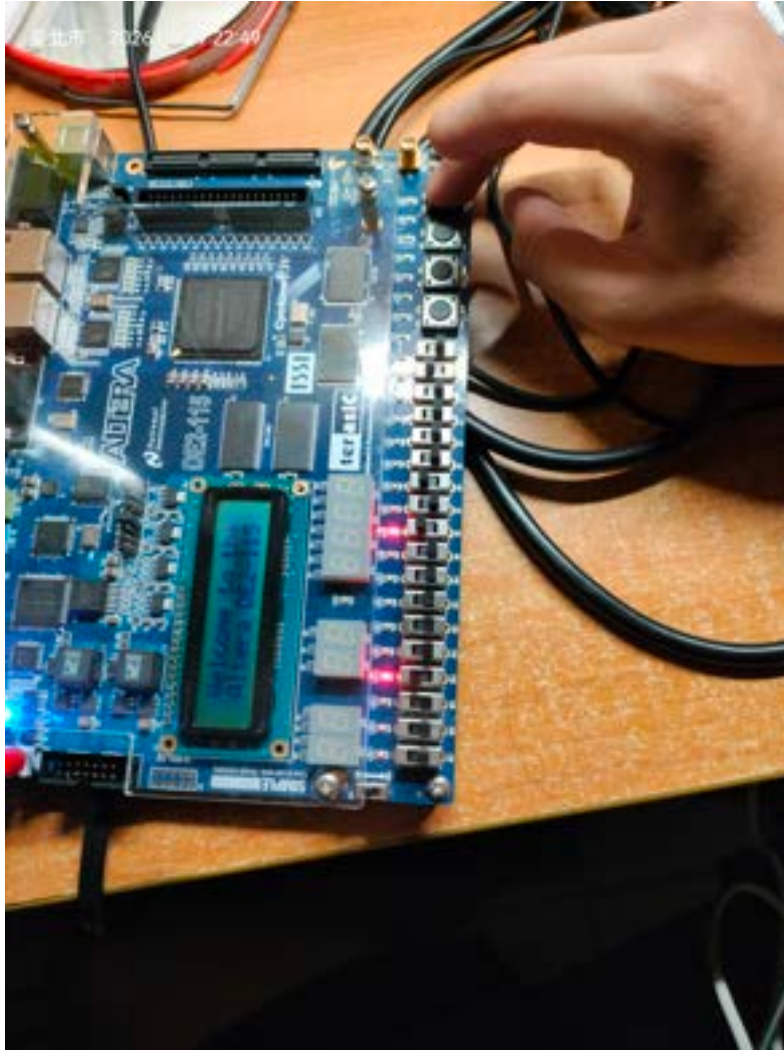


Figure 3: 按下 KEY[0] 第一次：FSM 從 Start 進入 S1，R 暫存器執行試減，LED 顯示目前餘數狀態。

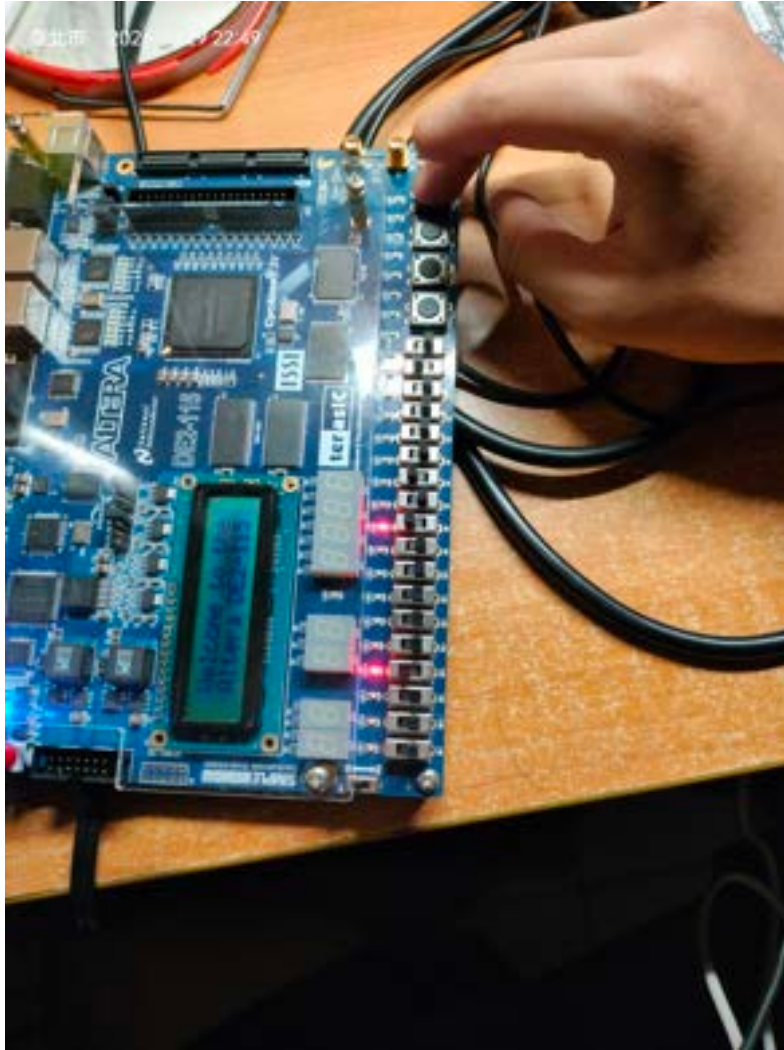


Figure 4: FSM 進入 S2a (餘數非負)：商數暫存器左移並串入'1'，LED 顯示商數中間運算過程。

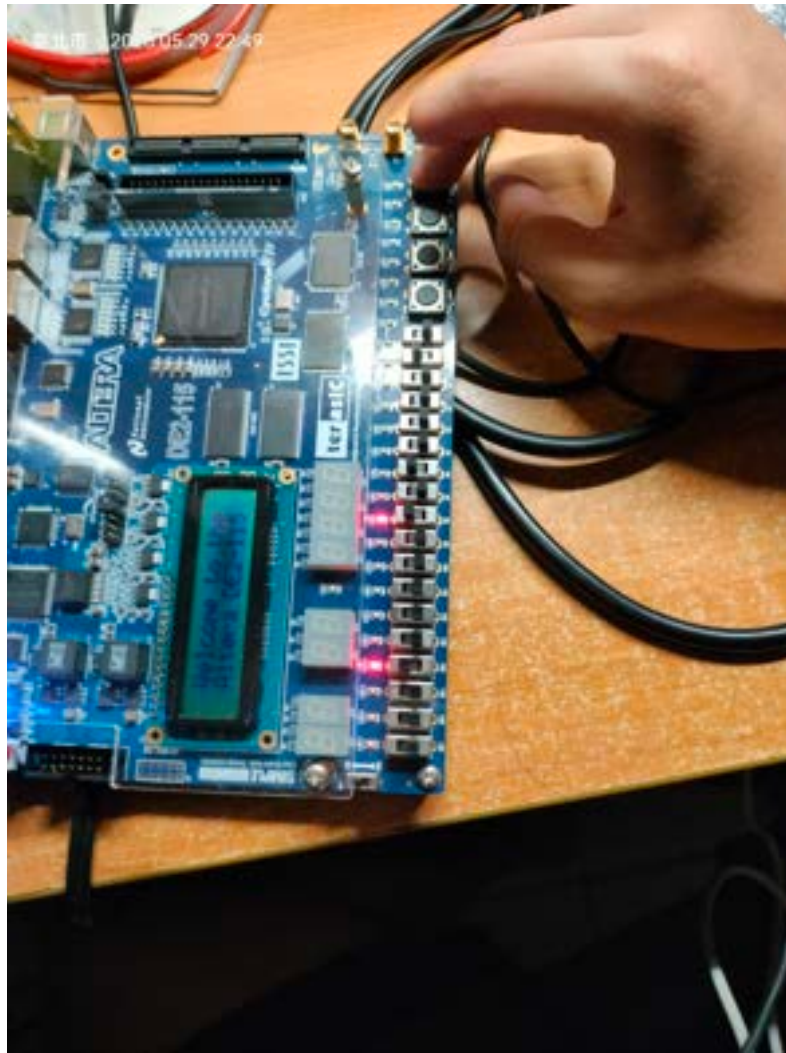


Figure 5: FSM 進入 S3：除數暫存器右移一位，計數器遞增，準備進行下一輪迭代。

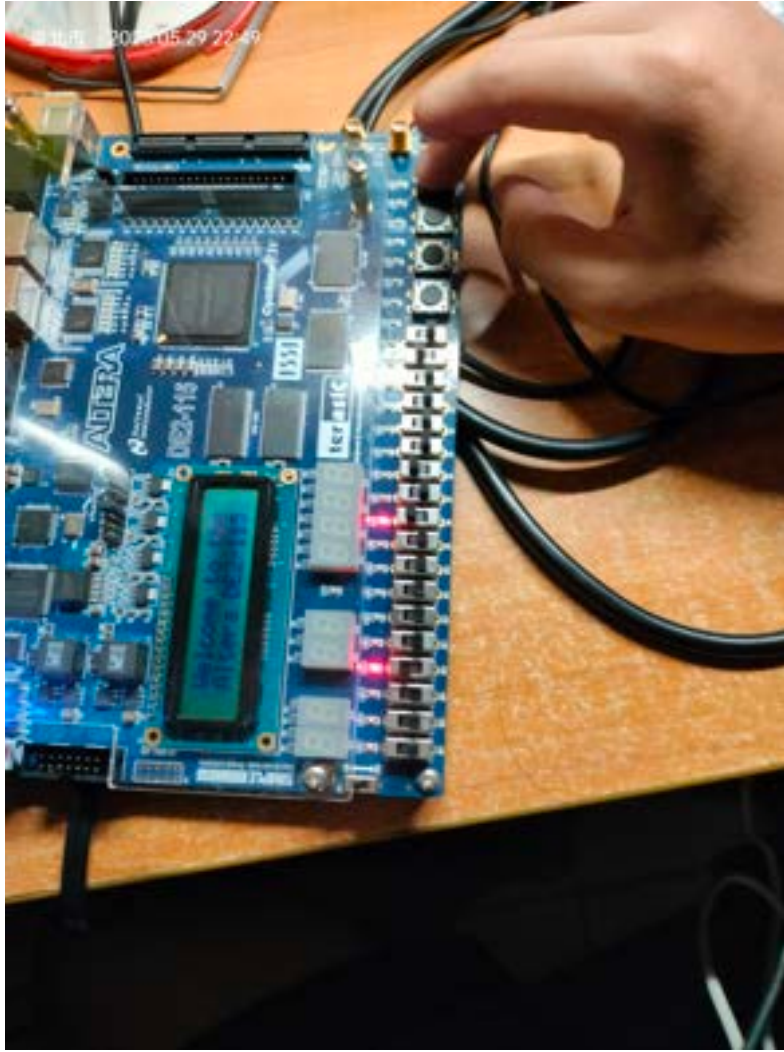


Figure 6: 迭代過程中 FSM 進入 S2b (餘數為負)：執行餘數恢復操作 ($R = R + D$)，商數左移串入 '0'。

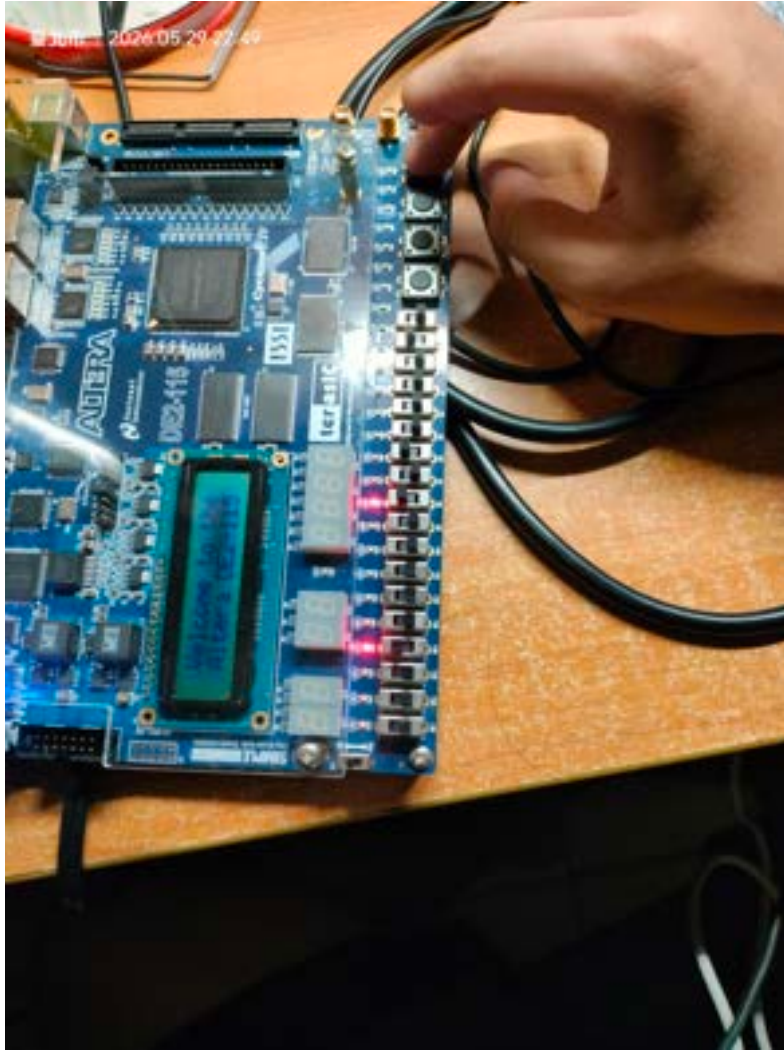


Figure 7: 持續按壓 KEY[0] 逐步執行迭代：LED 顯示各迭代步驟中商數與餘數的中間值。

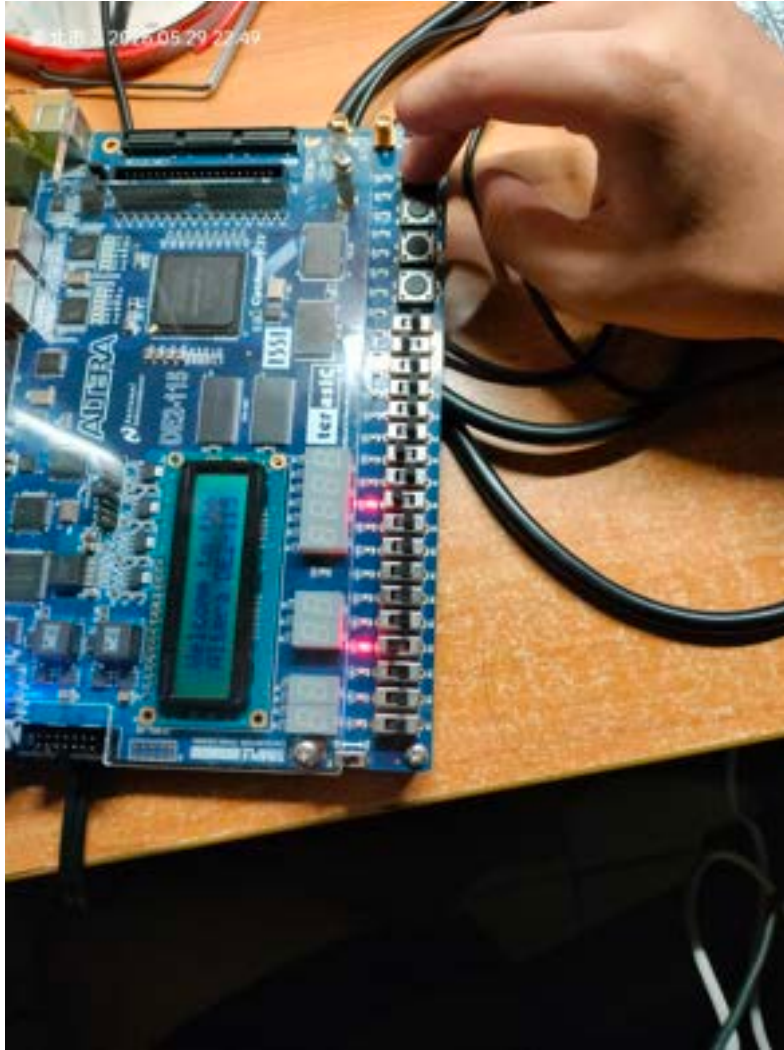


Figure 8: 第 8 次迭代中的 S1 狀態：FSM 執行最後一次試減操作。

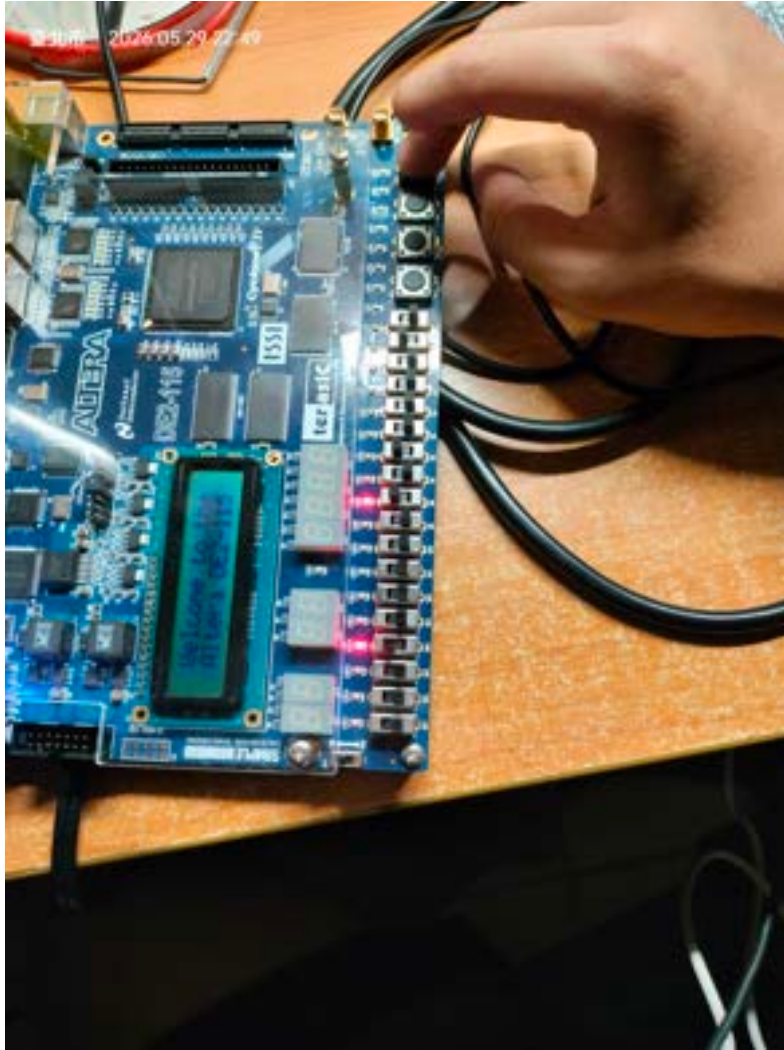


Figure 9: 第 9 次迭代後 S3 進入 S4 (count = 8) : FSM 凍結，運算完成。

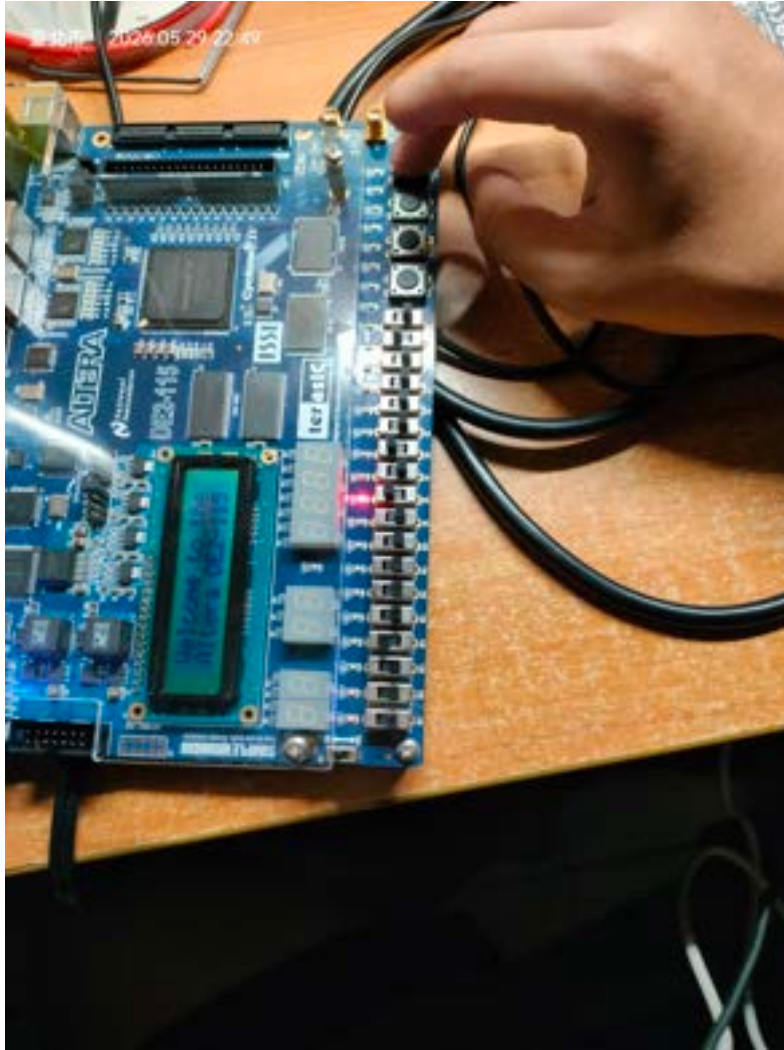


Figure 10: S4 完成狀態（一）：HEX1:HEX0 顯示最終商數，HEX3:HEX2 顯示最終餘數（十六進位格式）。

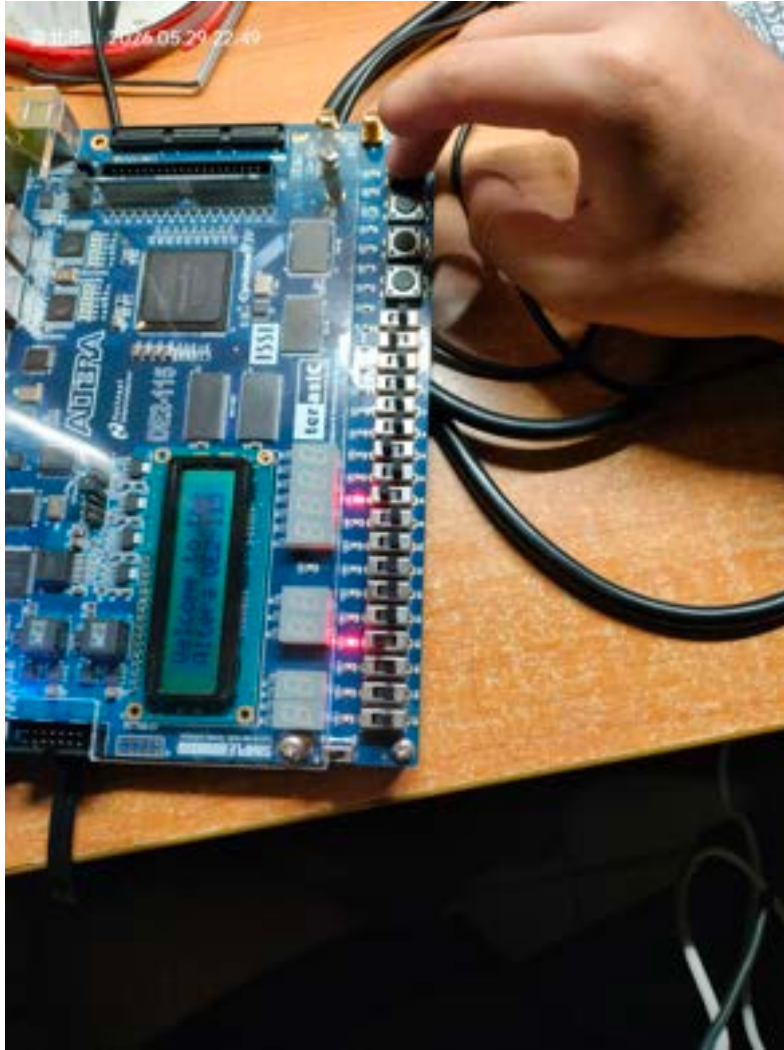


Figure 11: S4 完成狀態（二）：整體開發板正面照，確認七段顯示器顯示最終計算結果。

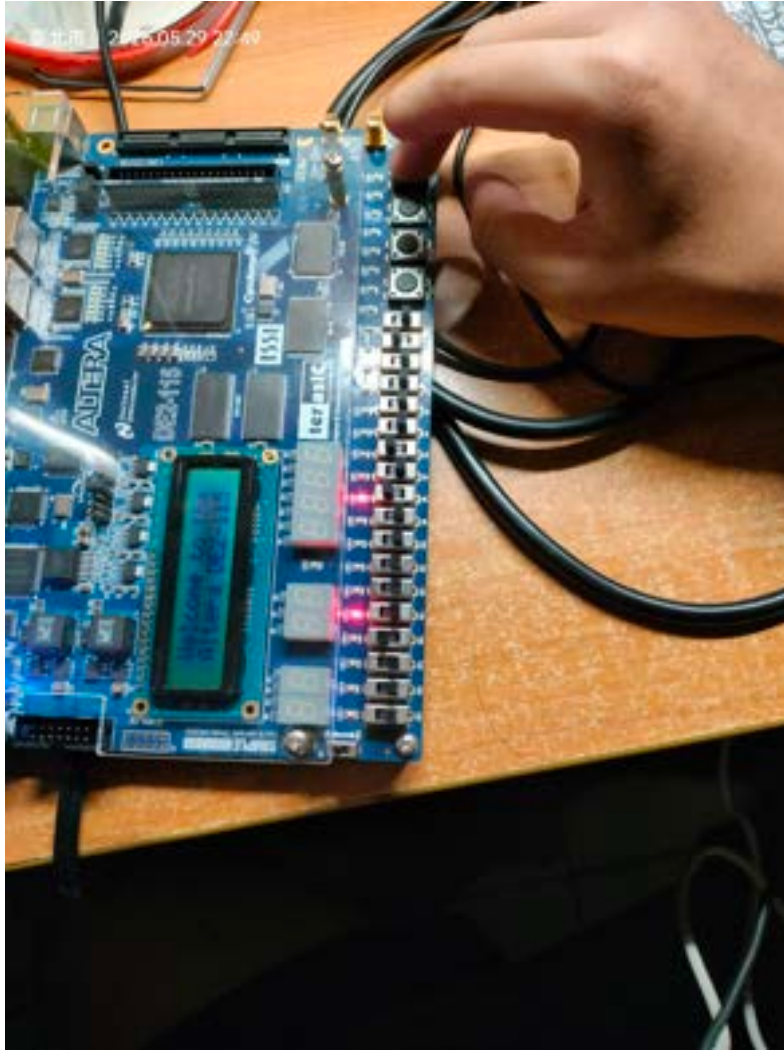


Figure 12: S4 完成狀態（三）：近距離確認四個七段顯示器上的數值正確無誤。

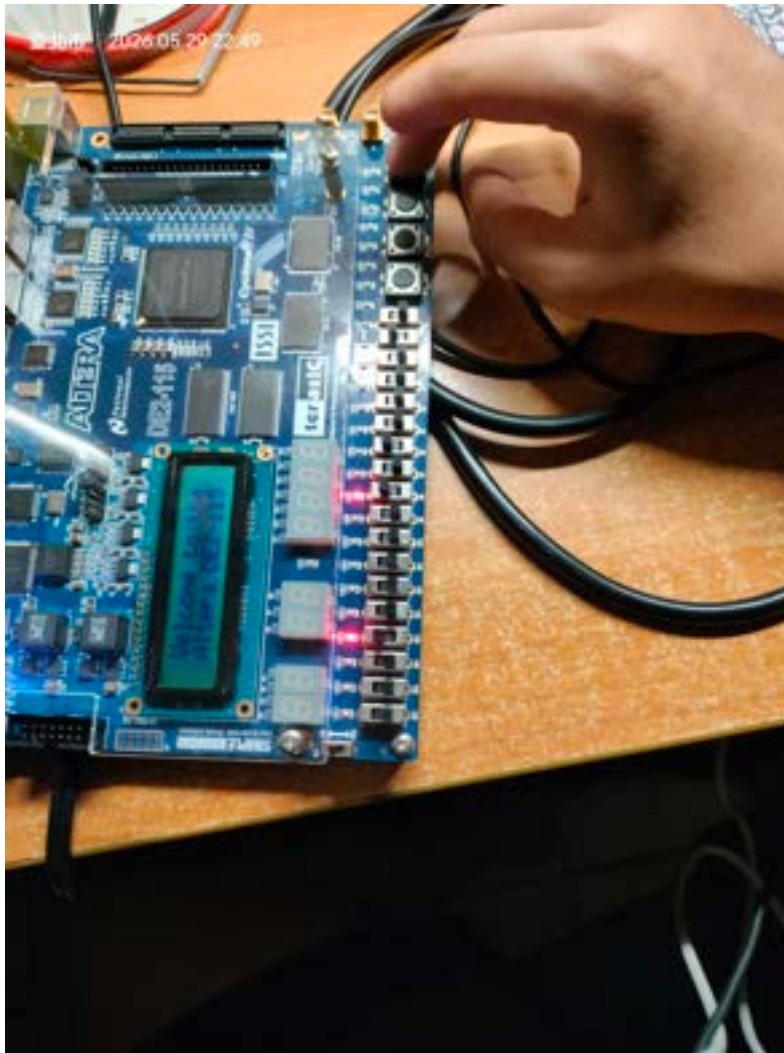


Figure 13: 驗證另一組輸入：更換 SW 撥碼開關設定，確認除法器對不同輸入值的運算正確性。

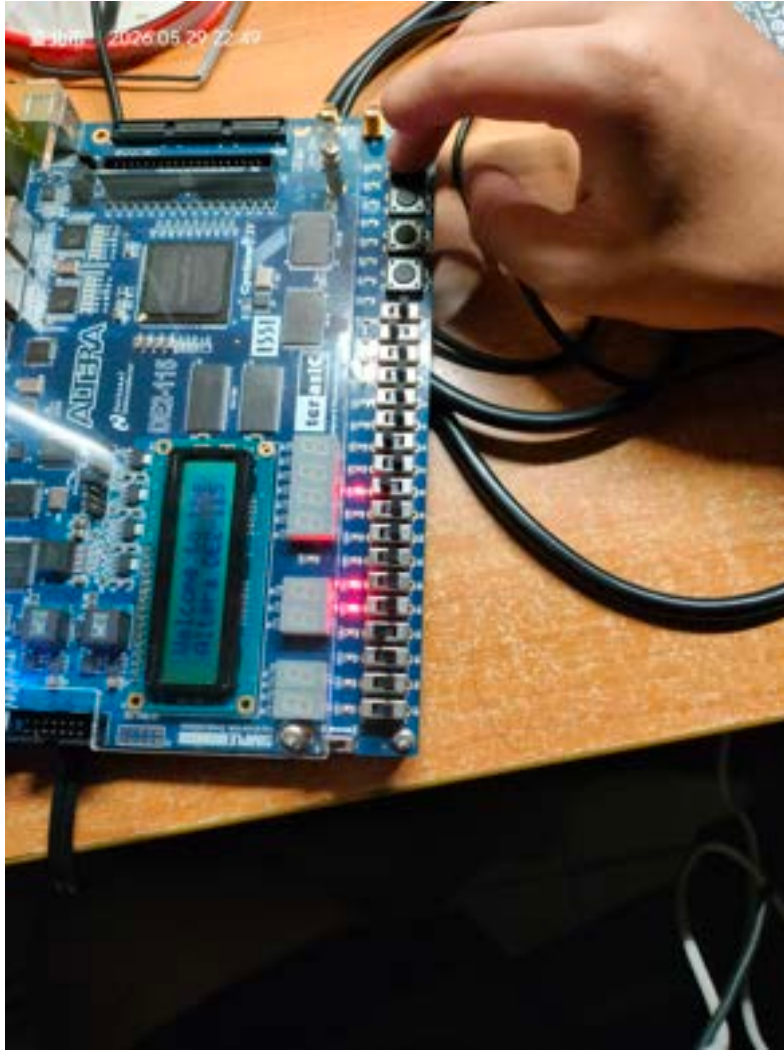


Figure 14: 重新 Reset 後再次運算（一）：SW[16] 撥高後，FSM 回到 Start 狀態，系統就緒。

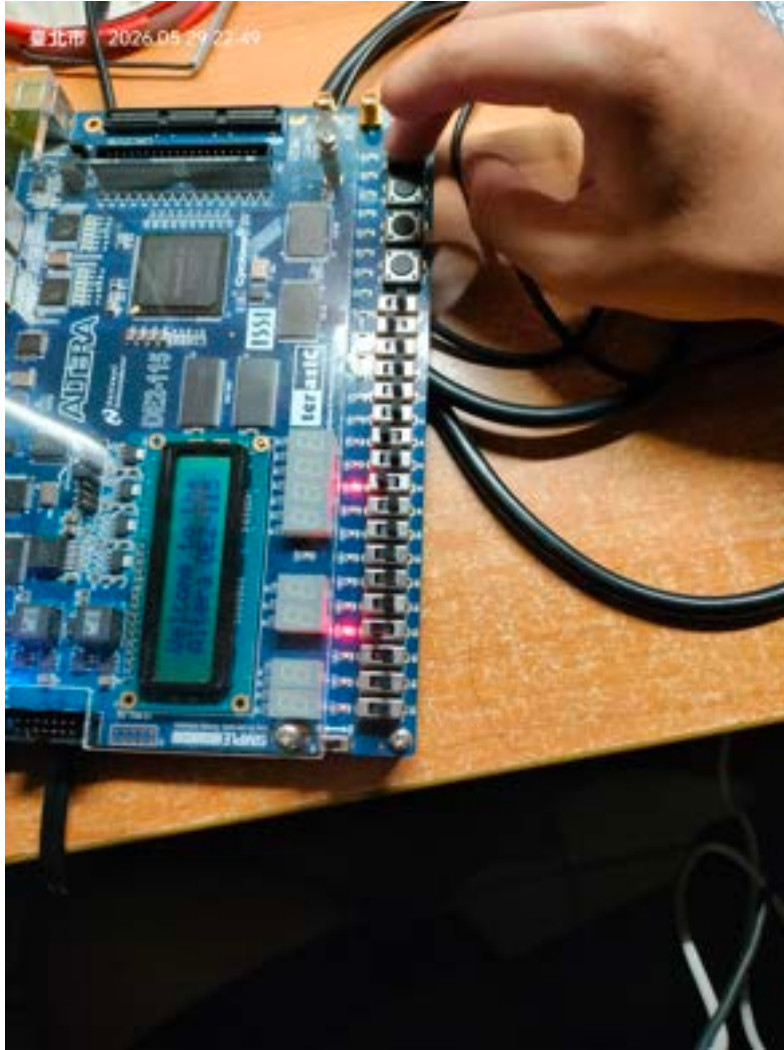


Figure 15: 重新 Reset 後再次運算（二）：設定新的被除數與除數，逐步驗證第二組測試案例。

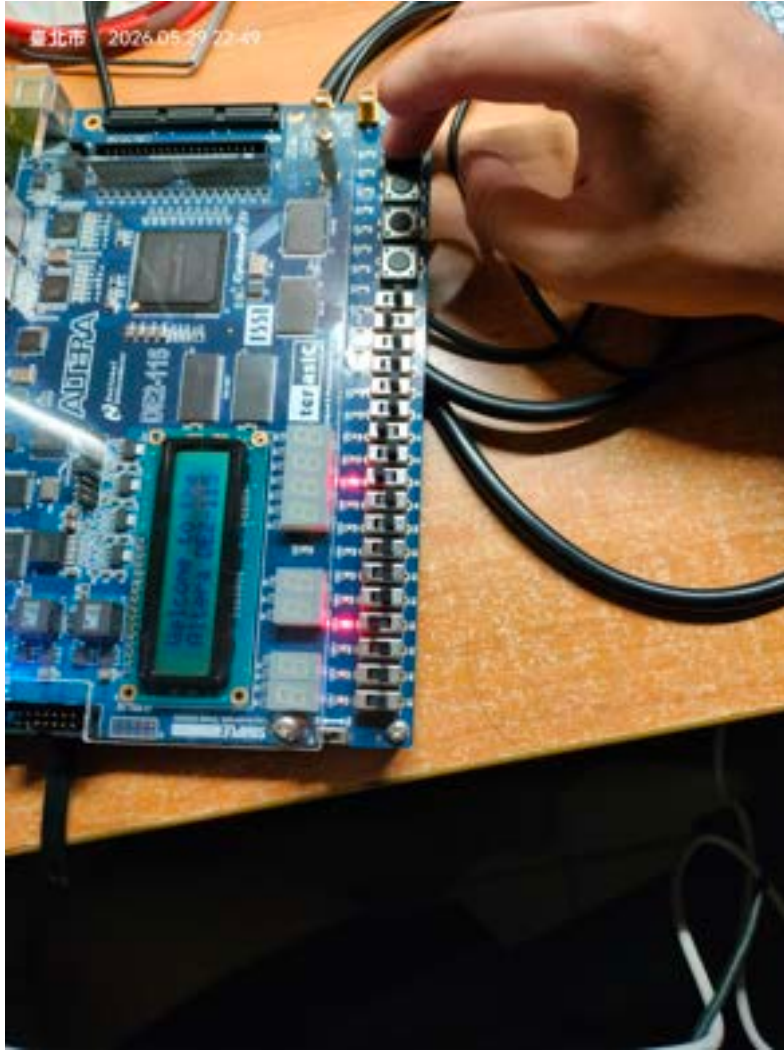


Figure 16: 第二組測試的中間迭代過程：LED 顯示不同輸入下各步驟的暫存器狀態。

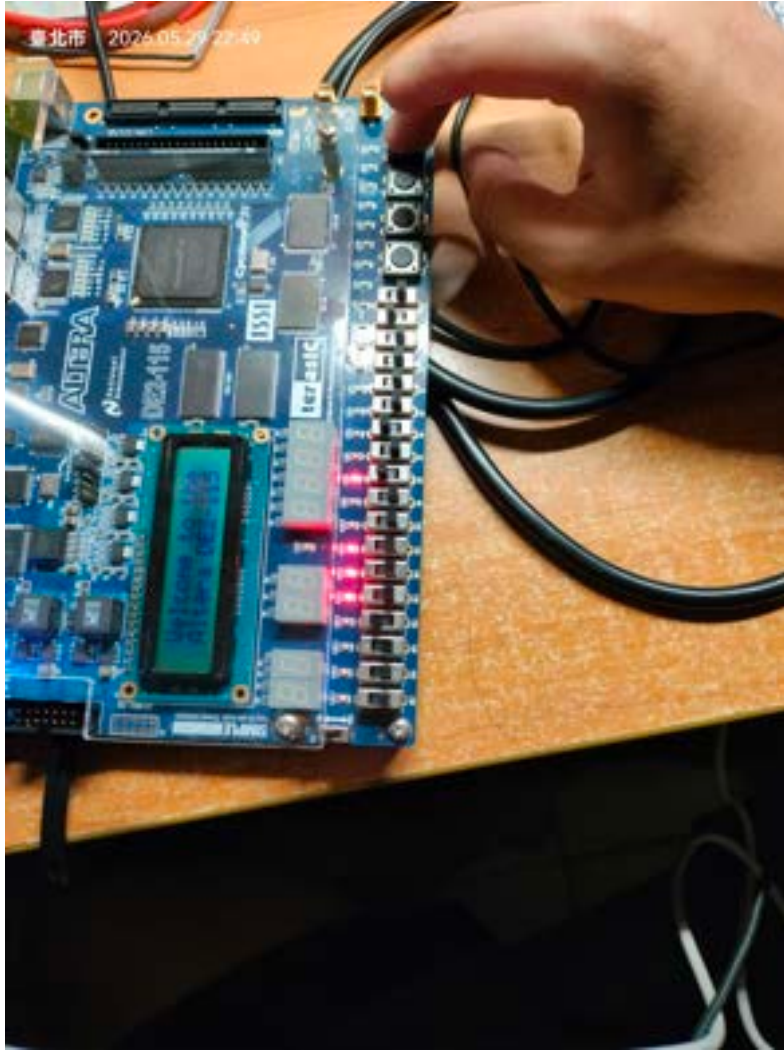


Figure 17: 第二組測試的 S4 結果（一）：七段顯示器顯示新輸入的最終商與餘數。

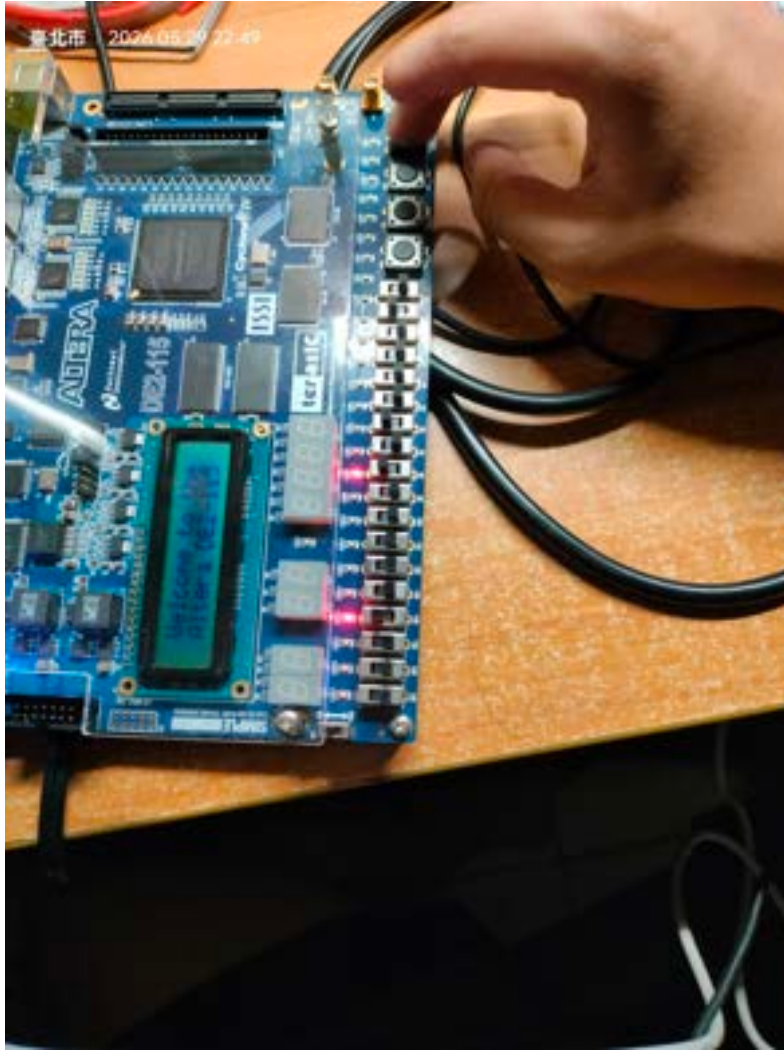


Figure 18: 第二組測試的 S4 結果（二）：完整板面照，確認 LED 與七段顯示器輸出一致。

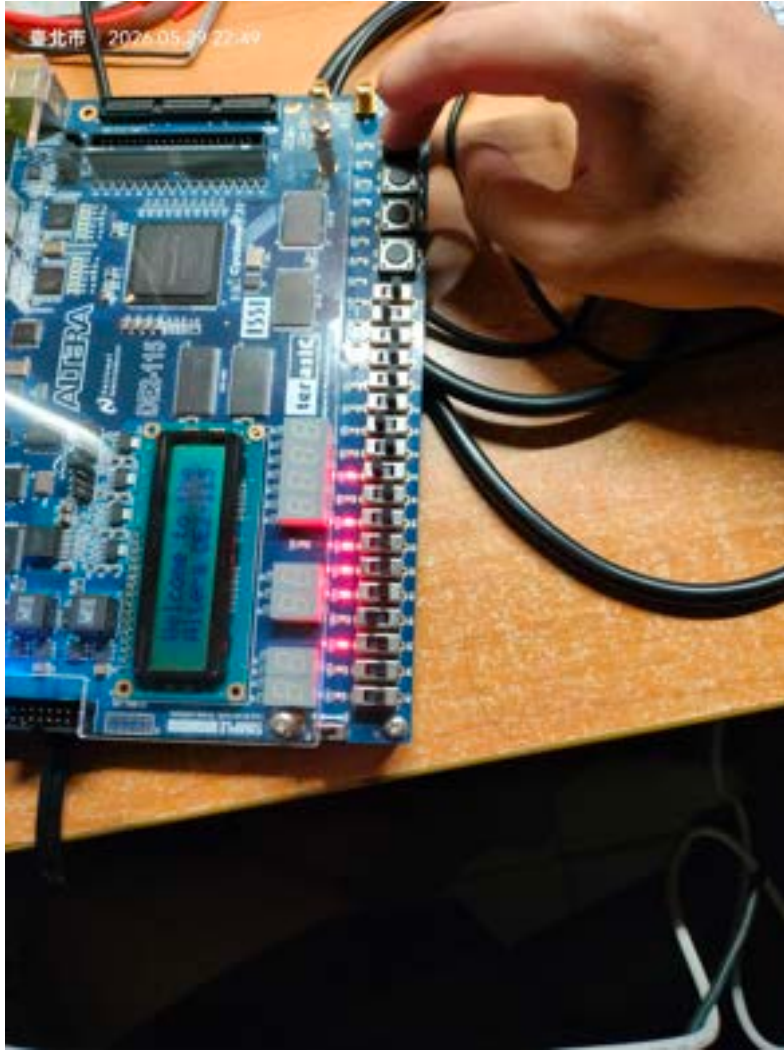


Figure 19: 特殊邊界值測試（一）：測試除數與被除數相等的特殊情況，驗證商為 1、餘數為 0。

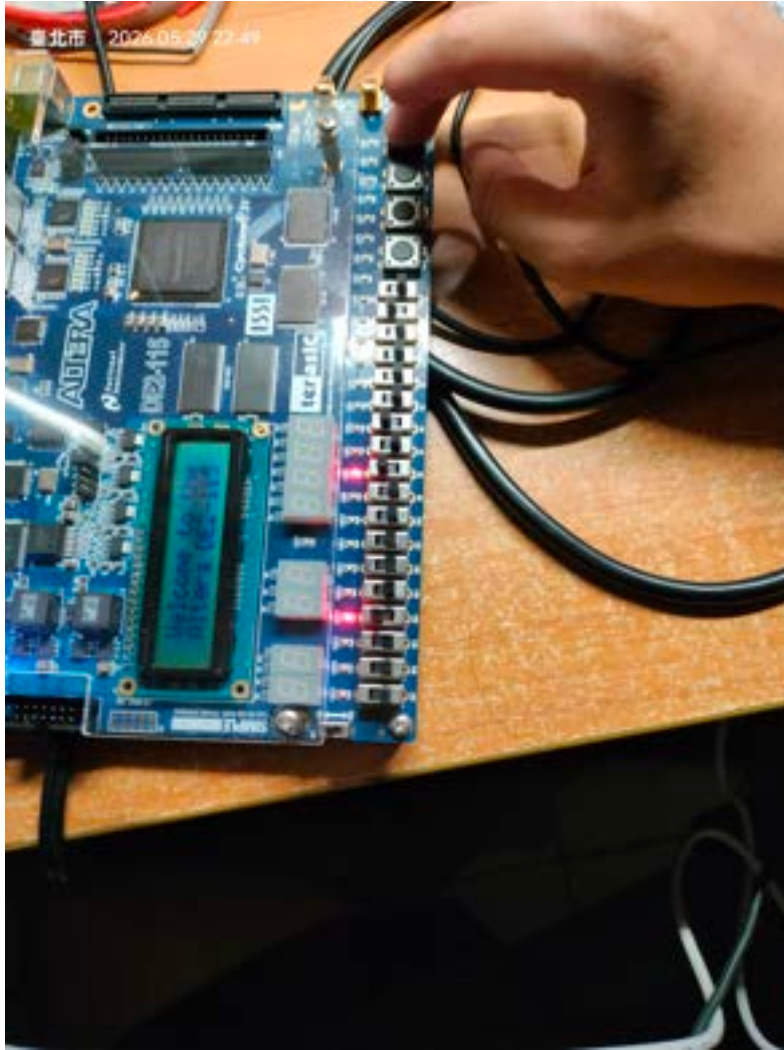


Figure 20: 特殊邊界值測試（二）：七段顯示器顯示 $\text{HEX1:HEX0} = 0100$ （商 4），驗證結果正確。

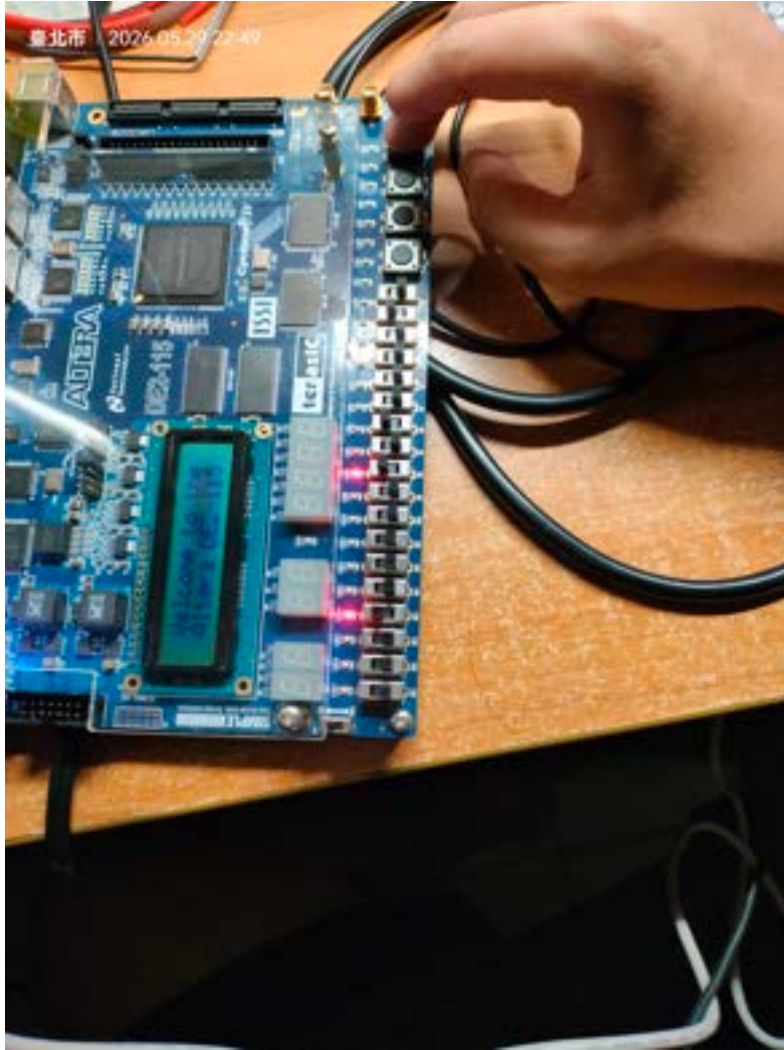


Figure 21: 整體驗證完成（近距離）：清晰可見七段顯示器上的最終商與餘數讀值。

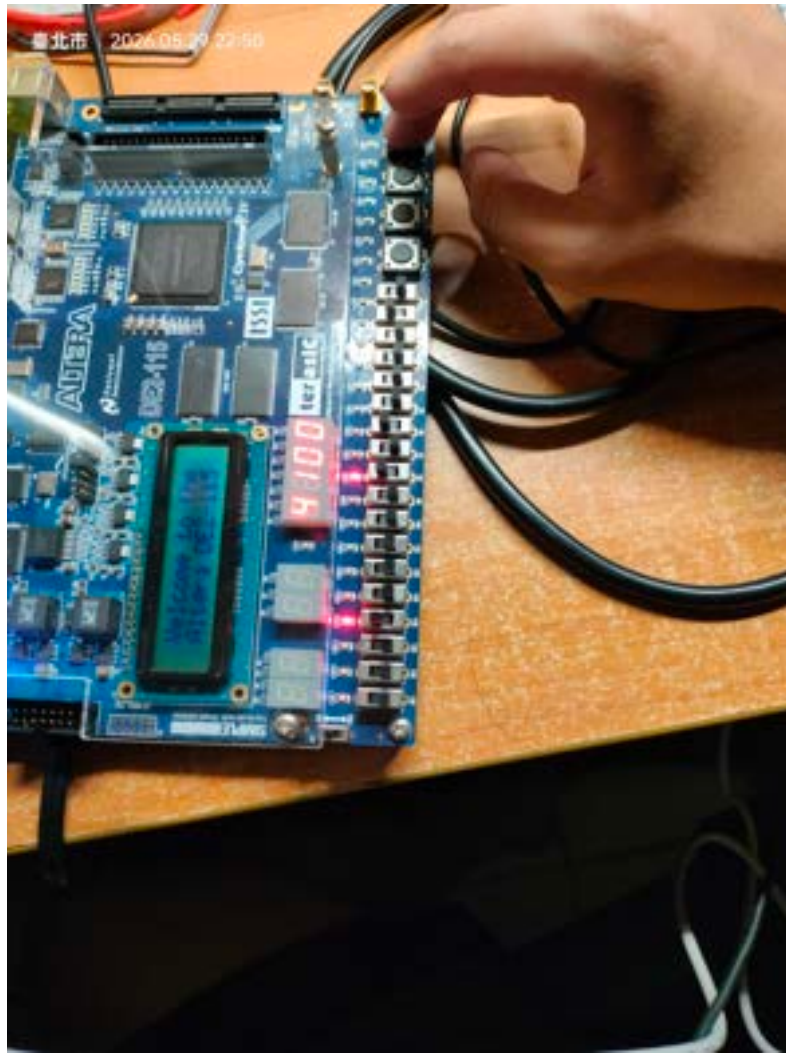


Figure 22: 最終驗證照（全板面）：開發板運作正常，LED 與七段顯示器均呈現正確計算結果，實驗圓滿完成。

4 實驗心得

在本次 Lab 07 的實驗中，我們以「結構化 (Structural)」架構設計了一個完整的 8 位元恢復除法器 (8-Bit Restoring Divider)。相較於 Lab 06 單純實作 FSM 控制器，本次實驗更全面地整合了資料路徑 (Datapath) 與控制器 (Controller) 兩大核心部分，是對 VHDL 硬體描述語言更深一層的實踐。

實驗最重要的設計亮點在於：直接重複使用 Lab 05 所設計的泛用移位暫存器 (N_bit_DFF)，透過 VHDL 的 `generic map` 機制，分別實例化出 8-bit 的商數暫存器 (Q)、以及 17-bit 的除數暫存器 (D) 與餘數暫存器 (R)。這讓我們深刻體會到硬體模組化設計的強大之處：一個設計良好的元件，能夠輕鬆因應不同的位元寬度需求，大幅提升設計的可重用性。

在恢復除法演算法的實作上，六個 FSM 狀態 (Start、S1、S2a、S2b、S3、S4) 與三個暫存器之間的交互控制是本次實驗最具挑戰性的部分。以 S1 的試減結果的符號位元 (`sub_res(16)`) 為判斷依據，動態決定是否需要「恢復」餘數 (S2b 的 $R = R + D$ 操作)，這個設計讓我們對於硬體如何實現條件判斷有了更具體的認識。

在硬體驗證階段，我們利用開發板上的 LED 與七段顯示器進行逐步驗證。由於 KEY[0] 作為手動步進時脈，我們能夠清楚地觀察到每個時脈觸發後各暫存器狀態的即時變化，這種直觀的驗證方式不僅加深了我們對演算法流程的理解，也讓我們成功確認了最終結果的正確性。

5 程式碼 (Division.vhd)

```
1  --
2  -- -----
3  -- Course: Microprocessor Systems (2026)
4  -- Laboratory: Lab 07 - 8-Bit Restoring Division Hardware Design (
5  -- Structural)
6  -- Author: Yang Cheng-Yu / Student ID: 113590051
7  -- File: Division.vhd
8  -- -----
9
10
11
12 library ieee;
13 use ieee.std_logic_1164.all;
14 use ieee.std_logic_unsigned.all;
15
16 entity Division is
17     port(
18         Clock      : in  std_logic;
19         Reset      : in  std_logic;
20         Divisor     : in  std_logic_vector(7 downto 0);
21         Dividend    : in  std_logic_vector(7 downto 0);
22         Quotient    : out std_logic_vector(7 downto 0);
23         Remainder   : out std_logic_vector(7 downto 0);
24         HEX0        : out std_logic_vector(6 downto 0);
25         HEX1        : out std_logic_vector(6 downto 0);
26         HEX2        : out std_logic_vector(6 downto 0);
27         HEX3        : out std_logic_vector(6 downto 0)
28     );
29 end entity Division;
30
31 architecture Structural of Division is
32
33     component n_bit_DFF is
34         generic( N : integer := 10 );
35         port(
36             clk      : in  std_logic;
37             clean    : in  std_logic;
38             load     : in  std_logic;
39             lr_sel   : in  std_logic;
40             di       : in  std_logic_vector(N-1 downto 0);
41             sdi      : in  std_logic;
42             qo       : out std_logic_vector(N-1 downto 0)
43         );
44     end component;
45
46     type state_type is (Start, S1, S2a, S2b, S3, S4);
47     signal current_state : state_type := Start;
48     signal count         : integer range 0 to 9 := 0;
49
50     signal clean_Q, load_Q, lr_sel_Q, sdi_Q : std_logic;
51     signal di_Q : std_logic_vector(7 downto 0);
52     signal qo_Q : std_logic_vector(7 downto 0);
53
54     signal clean_D, load_D, lr_sel_D, sdi_D : std_logic;
```

```

51 signal di_D : std_logic_vector(16 downto 0);
52 signal qo_D : std_logic_vector(16 downto 0);
53
54 signal clean_R, load_R, lr_sel_R, sdi_R : std_logic;
55 signal di_R : std_logic_vector(16 downto 0);
56 signal qo_R : std_logic_vector(16 downto 0);
57
58 signal sub_res : std_logic_vector(16 downto 0);
59
60 function to_seven_seg(nibble : std_logic_vector(3 downto 0))
61     return std_logic_vector is
62 begin
63     case nibble is
64         when "0000" => return "1000000"; -- 0
65         when "0001" => return "1111001"; -- 1
66         when "0010" => return "0100100"; -- 2
67         when "0011" => return "0110000"; -- 3
68         when "0100" => return "0011001"; -- 4
69         when "0101" => return "0010010"; -- 5
70         when "0110" => return "0000010"; -- 6
71         when "0111" => return "1111000"; -- 7
72         when "1000" => return "0000000"; -- 8
73         when "1001" => return "0010000"; -- 9
74         when "1010" => return "0001000"; -- A
75         when "1011" => return "0000011"; -- b
76         when "1100" => return "1000110"; -- C
77         when "1101" => return "0100001"; -- d
78         when "1110" => return "0000110"; -- E
79         when "1111" => return "0001110"; -- F
80         when others => return "1111111"; -- Blank
81     end case;
82 end function;
83
84 begin
85
86 reg_Q_inst : n_bit_DFF
87     generic map( N => 8 )
88     port map( clk=>Clock, clean=>clean_Q, load=>load_Q,
89             lr_sel=>lr_sel_Q, di=>di_Q, sdi=>sdi_Q, qo=>qo_Q );
90
91 reg_D_inst : n_bit_DFF
92     generic map( N => 17 )
93     port map( clk=>Clock, clean=>clean_D, load=>load_D,
94             lr_sel=>lr_sel_D, di=>di_D, sdi=>sdi_D, qo=>qo_D );
95
96 reg_R_inst : n_bit_DFF
97     generic map( N => 17 )
98     port map( clk=>Clock, clean=>clean_R, load=>load_R,
99             lr_sel=>lr_sel_R, di=>di_R, sdi=>sdi_R, qo=>qo_R );
100
101 sub_res    <= qo_R - qo_D;
102 Quotient   <= qo_Q;
103 Remainder  <= qo_R(7 downto 0);
104
105 process(current_state, Dividend, Divisor, qo_Q, qo_D, qo_R, sub_res)
106 begin
107     clean_Q<='0'; load_Q<='1'; lr_sel_Q<='1'; sdi_Q<='0'; di_Q<=qo_Q;
108     clean_D<='0'; load_D<='1'; lr_sel_D<='0'; sdi_D<='0'; di_D<=qo_D;

```

```

109     clean_R<='0'; load_R<='1'; lr_sel_R<='1'; sdi_R<='0'; di_R<=qo_R;
110
111     case current_state is
112         when Start =>
113             clean_Q <= '1'; load_Q <= '0';
114             load_D  <= '1'; di_D  <= '0' & Divisor & "00000000";
115             load_R  <= '1'; di_R  <= "00000000" & Dividend;
116         when S1 =>
117             load_R <= '1'; di_R <= sub_res;
118         when S2a =>
119             load_Q <= '0'; lr_sel_Q <= '1'; sdi_Q <= '1';
120         when S2b =>
121             load_R <= '1'; di_R <= qo_R + qo_D;
122             load_Q <= '0'; lr_sel_Q <= '1'; sdi_Q <= '0';
123         when S3 =>
124             load_D <= '0'; lr_sel_D <= '0'; sdi_D <= '0';
125         when S4 =>
126             null;
127     end case;
128 end process;
129
130 process(Clock, Reset)
131 begin
132     if Reset = '1' then
133         current_state <= Start; count <= 0;
134     elsif rising_edge(Clock) then
135         case current_state is
136             when Start => count <= 0; current_state <= S1;
137             when S1 =>
138                 if sub_res(16) = '0' then current_state <= S2a;
139                 else current_state <= S2b; end if
140 ;
141                 when S2a => current_state <= S3;
142                 when S2b => current_state <= S3;
143                 when S3 =>
144                     if count = 8 then current_state <= S4;
145                     else count <= count + 1; current_state <= S1; end if;
146                 when S4 => current_state <= S4;
147             end case;
148         end if;
149     end process;
150
151 process(current_state, qo_Q, qo_R)
152 begin
153     if current_state = S4 then
154         HEX0 <= to_seven_seg(qo_Q(3 downto 0));
155         HEX1 <= to_seven_seg(qo_Q(7 downto 4));
156         HEX2 <= to_seven_seg(qo_R(3 downto 0));
157         HEX3 <= to_seven_seg(qo_R(7 downto 4));
158     else
159         HEX0 <= (others => '1'); HEX1 <= (others => '1');
160         HEX2 <= (others => '1'); HEX3 <= (others => '1');
161     end if;
162 end process;
163 end architecture Structural;

```

Listing 1: Division.vhd — 8-bit 恢復除法器頂層模組（結構化架構）

6 程式碼 (N_bit_DFF.vhd)

```
1  --  
2  -----  
3  -- Course: Microprocessor Systems (2026)  
4  -- Laboratory: Lab 07 - Multi-Purpose Shift Register (Reused from Lab 05)  
5  -- File: N_bit_DFF.vhd  
6  --  
7  -----  
8  
9  
10  
11 library ieee;  
12 use ieee.std_logic_1164.all;  
13 use ieee.std_logic_unsigned.all;  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
entity n_bit_DFF is  
    generic( N : integer := 10 );  
    port(  
        clk      : in  std_logic;  
        clean    : in  std_logic;  
        load     : in  std_logic;  
        lr_sel   : in  std_logic;  
        di       : in  std_logic_vector(N-1 downto 0);  
        sdi      : in  std_logic;  
        qo       : out std_logic_vector(N-1 downto 0)  
    );  
end entity n_bit_DFF;  
  
architecture Behavioral of n_bit_DFF is  
    signal reg : std_logic_vector(N-1 downto 0) := (others => '0');  
begin  
    process(clk)  
    begin  
        if rising_edge(clk) then  
            if clean = '1' then  
                reg <= (others => '0');  
            elsif load = '1' then  
                reg <= di;  
            elsif lr_sel = '1' then  
                -- Left shift: reg(N-1) <- reg(N-2) <- ... <- reg(0) <- sdi  
                for i in N-1 downto 1 loop  
                    reg(i) <= reg(i-1);  
                end loop;  
                reg(0) <= sdi;  
            elsif lr_sel = '0' then  
                -- Right shift: reg(0) -> reg(1) -> ... -> reg(N-1) -> sdi  
                for i in 0 to N-2 loop  
                    reg(i) <= reg(i+1);  
                end loop;  
                reg(N-1) <= sdi;  
            end if;  
        end if;  
    end process;  
  
    qo <= reg;
```

```
52 end architecture Behavioral;
```

Listing 2: N_bit_DFF.vhd — 泛用 N-bit 移位暫存器元件（重用自 Lab 05）